Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №1

«Множини: основні властивості та операції над ними, діаграми Венна».

Виконав:
студент групи ІО-45
Унятицький А.Д.
Перевірив Пономаренко А.М.

Київ – 2025

Лабораторна робота №1

Тема: «Множини: основні властивості та операції над ними, діаграми Венна».

Мета: вивчити основні аксіоми, закони і теореми теорії множин, навчитися застосовувати їх на практиці. Обчислити логічний вираз шляхом послідовного застосування операцій над множинами.

Загальне завдання:

1. Повторити матеріал: «Бібліотека tkinter (віджети)» та виконати лабораторну роботу з застосуванням графічного інтерфейсу.
2. Спростити логічний вираз з застосуванням тотожностей алгебри множин.
3. В окремому модулі написати функцію обчислення початкового логічного виразу (1), вибраного відповідно до індивідуального варіанта.
4. В окремому модулі написати функцію обчислення спрощеного логічного виразу.
5. В окремому модулі написати функцію виконання логічної операції (2), вибраної відповідно до індивідуального варіанта.
6. В окремому модулі виконати порівняння результатів:

А) обчислення початкового та спрощеного виразу

Б) виконання логічної операції Вашою функцією та відповідною стандартною логічною операцією або функцією Python.

Теоретичні основи:

1.1. Властивості множин

Множина – є сукупність визначених об'єктів, різних між собою, об’єднаних за певною ознакою чи властивістю.

Множини позначають великими латинськими буквами. Об'єкти, що складають множини, називають елементами і позначають малими буквами латинського алфавіту.

На практиці часто застосовують такі позначення:

N – множина натуральних чисел;

P – множина додатних цілих чисел;

Z – множина додатних і від’ємних цілих чисел, включаючи нуль;

Q – множина раціональних чисел;

R – множина дійсних чисел.

Якщо множина не містить жодного елемента, її називають порожньою і позначають Ø.

Скінченна множина – це така множина, кількість елементів якої може бути виражена скінченним числом, причому не важливо, чи можемо ми порахувати це число в даний момент.

Нескінченна множина – це така множина, що не є скінченною.

1.2 Операції над множинами

Об'єднання. Об'єднанням множин A і B називають множину, що складається із всіх тих елементів, які належать хоча б одній з множин A або B . Об'єднання множин A і B позначають A ∪ B.

Перетин. Перетином множин A і B називають множину, що складається із всіх тих елементів, які належать як множині A, так і множині B . Перетин множин A і B позначають A ∩ B.

Доповнення. Доповненням (або абсолютним доповненням) множини A називають множину, що складається із всіх елементів універсальної множини, які не належать A. Доповнення множини A позначають A .

Різниця. Різницею множин A й B (або відносним доповненням) називають множину, що складається із всіх елементів множини A, які не належать B . Різницю множин A і B позначають A− B або A \ B.

1.3 Діаграми Венна

Для графічної ілюстрації операцій над множинами даної універсальної множини U використовують діаграми Венна. Діаграма Венна – це зображення множини у вигляді геометричної множини, наприклад, кола. При цьому універсальну множину зображують у вигляді прямокутника.

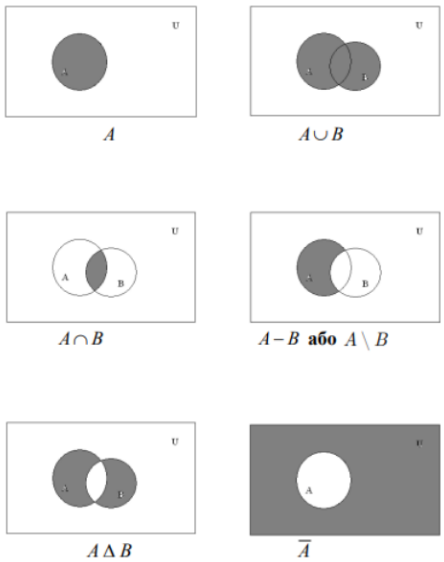


Рис. 1.2.

1.4. Тотожності алгебри множин

1. Комутативність об'єднання $A \cup B = B \cup A$	1. Комутативність перетину $A \cap B = B \cap A$
2. Асоціативність об'єднання $A \cup (B \cup C) = (A \cup B) \cup C$	2. Асоціативність перетину $A \cap (B \cap C) = (A \cap B) \cap C$
3. Дистрибутивність об'єднання відносно перетину $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	3. Дистрибутивність перетину відносно об'єднання $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
4. Закони дій з пустою та універсальною множинами $A \cup \varnothing = A$ $A \cup \overline{A} = U$ $A \cup U = U$	4. Закони дій з пустою та універсальною множинами $A \cap U = A$ $A \cap \overline{A} = \varnothing$ $A \cap \varnothing = \varnothing$
5.Закон ідемпотентності об'єднання Термін **ідемпотентність** означує властивість математичного об'єкта, яка проявляється в тому, що повторна дія над об'єктом *не змінює* його $A \cup A = A$	5. Закон ідемпотентності перетину $A \cap A = A$
6. Закон де Моргана $\overline{A \cup B} = \overline{A} \cap \overline{B}$	6. Закон де Моргана $\overline{A \cap B} = \overline{A} \cup \overline{B}$
7. Закон поглинання $A \cup (A \cap B) = A$	7. Закон поглинання $A \cap (A \cup B) = A$
8. Закон склеювання $(A \cap B) \cup (A \cap \overline{B}) = A$	8. Закон склеювання $(A \cup B) \cap (A \cup \overline{B}) = A$
9. Закон Порецького $A \cup (\overline{A} \cap B) = A \cup B$	9. Закон Порецького $A \cap (\overline{A} \cup B) = A \cap B$
10. Закон подвійного доповнення $\overline{\overline{A}} = A$	
11. Визначення операції «різниця»: $A \setminus B = A \cap \overline{B}$.	
12. Визначення операції «симетрична різниця»: $A \Delta B = (A \setminus B) \cup (B \setminus A)$.	

1.6 Загальний порядок виконання лабораторної роботи

А) Вибрати номер Z індивідуального варіанта відповідно до виразу:

Z=(i+G%60)%30+1, де i – номер у списку групи, G – числова складова назви групи.

Б) Максимально спростити логічний вираз. Для спрощення використати тотожності алгебри множин та визначення логічних операцій. Спрощення є

максимальним, якщо формула містить тільки одне входження кожної множини.

В) Створити блок-схему послідовності обчислення початкового логічного виразу.

Г) Створити блок-схему послідовності обчислення спрощеного логічного виразу.

Д) З використанням блок-схем створити проект, який містить модуль з функцією для обчислення початкового виразу (1) та модуль з функцією для обчислення спрощеного виразу.

Е) З використанням блок-схем, заданих у лабораторній роботі, створити модуль з функцією для виконання логічної операції (2), вибраної відповідно до варіанта.

Ж) В основному файлі виконати перевірку правильності спрощення виразу з виводом відповідного повідомлення.

З) Як елементи множин можуть бути використані числа від 0 до 255.

І) Лабораторну роботу виконувати з застосуванням мови Python та бібліотеки tkinter.

1.7 Вимоги до інтерфейсу

А) Програма повинна складатися з 5-ти вікон.

Вміст вікна №1

1. Головне меню, яке повинно включати виклик вікна №2, вікна №3, вікна №4 та вікна №5.

2. Віджети виводу П.І.Б студента, номера групи, номера у списку та віджет виводу результатів обчислення варіанту відповідно да програми, що задана у завданні – пункт (А) загального порядку виконання лабораторної роботи).

3. Віджети для задавання потужності множин A, B і C.

4. Віджети для формування випадковим чином множин A,B і C заданої потужності.

5. Віджети, що дають можливість ручного вводу множин A,B і C.

6. Віджет для задавання діапазону цілих чисел, які будемо вважати універсальною множиною.

Вміст вікна №2

1. Віджети для відображення елементів множин A, B і C.

2. Віджети запуску покрокового виконання початкового виразу (1).

Одним кроком вважати виконання однієї логічної операції.

3. Віджети відображення множин-операндів та множини-результату кожної логічної операції.

4. Віджет відображення множини D та віджет для виконання команди збереження даного результату у файлі.

Вміст вікна №3

1. Віджети для відображення елементів множин A, B і C.

2. Віджети запуску покрокового виконання спрощеного логічного виразу.

Одним кроком вважати виконання однієї логічної операції.

3. Віджети відображення множин-операндів та множини-результату кожної логічної операції.

4. Віджет відображення множини D, та віджет для виконання команди збереження даного результату у файлі.

Вміст вікна №4

1. Віджети для відображення елементів множин X і Y.

2. Віджет відображення результату Z виконання заданої у індивідуальному варіанті логічної операції (2) над множинами X і Y, яка виконана за допомогою написаної вами функції.

Вміст вікна №5

1. Віджети для зчитування з файлу та відображення множини, яка є результатом обчислення початкового виразу.

2. Віджети для зчитування з файлу та відображення множини, яка є результатом обчислення спрощеного виразу.

3. Віджети для зчитування з файлу та відображення множини Z, яка є результатом

обчислення операції над множинами X і Y, яка виконана за допомогою написаної вами функції.

4. Віджети для запуску та відображення логічної операції над множинами X і Y, яка є результатом обчислення Z з використанням стандартної функції Python.

5. Віджети для порівняння результатів виводу у пунктах 1 та 2 та у пунктах 3 та 4

Виконання індивідуальних завдань

Программа обчислення варіанта:

```
Моя група - ІО-45
Мій номер у групі - 23
Мій варіант - 9
```

9	(1)	$D = \left(\left(\left(A \cup \bar{A}\right) \cap A\right) \setminus B \cup B\right) \cap \overline{\left(C \cup \left(C \cap B\right)\right)}$
	(2)	$X = A; Y = C; Z = X \setminus Y$

Спрощення логічного виразу

D = $(((A \cup \bar{A}) \cap A) \setminus (B \cup B)) \cap \overline{(C \cup (C \cap B))}$

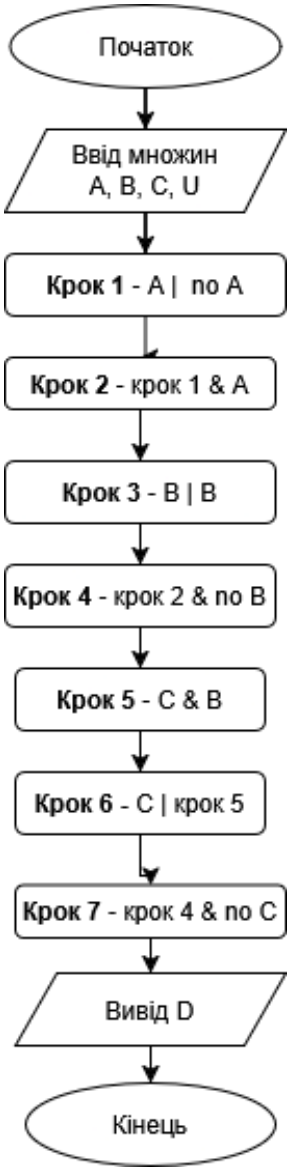
D = $((U \cap A) \setminus (B \cup B)) \cap \overline{(C \cup (C \cap B))}$

D = $(A \setminus B) \cap \overline{(C \cup (C \cap B))}$

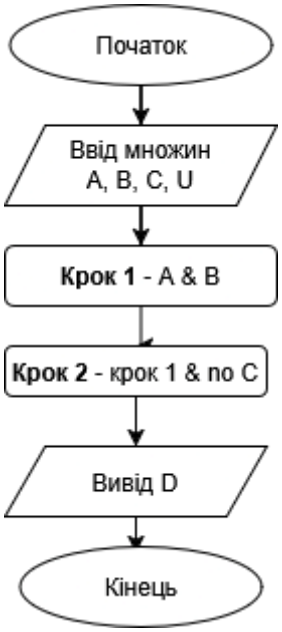
D = $(A \setminus B) \cap \overline{C}$

D = $A \cap \overline{B} \cap \overline{C}$

Блоксхема обчислення початкового логічного виразу



Блоксхема обчислення спрощеного логічного виразу



Код

```
# Модуль func_for_window2
# Функції обчислення початкового логічного виразу

def not_a_f(u, a):
    return u - a

def not_b_f(u, b):
    return u - b

def not_c_f(u, c):
    not_c = u - c
    return not_c

def step_1_f (a, not_a):
    step_1 = a | not_a
    return step_1

def step_2_f(step_1, a):
    step_2 = step_1 & a
    return step_2

def step_3_f(b):
    step_3 = b | b
    return step_3

def step_4_f(step_2, not_b):
    step_4 = step_2 & not_b
    return step_4

def step_5_f(c, b):
    step_5 = c & b
    return step_5

def step_6_f(c, step_5):
    step_6 = c | step_5
    return step_6

def step_7_f(step_4, not_c):
    step_7 = step_4 & not_c
    return step_7
```

```
# Модуль func_for_window3
# Функції для виконання спрощеного виразу
```

```
def not_c_f(u, c):
    return u - c

def not_b_f(u, b):
    return u -b

def step1_f_sh(a, not_b):
    step1_sh = a & not_b
    return step1_sh

def step2_f_sh(step1_sh, not_c):
    step2_sh = step1_sh & not_c
    return step2_sh
```

Модуль func_for_window4
Функції виконання логічної операції

```
def my_func(x, y, u):
    x = list(x)
    y = list(y)
    u = list(u)
    n_b = []
    for elem in u:
        if elem not in y:
            n_b.append(elem)
    c = []
    for elem in x:
        if elem in n_b:
            c.append(elem)
    return set(c)

def func_python(a, b, u):
    c = a & (u - b)
    return c
```

Модуль compare_rezult

```
def compare(file_long, file_short):
    if file_long == file_short:
        return "Результати D однакові"
    else:
        return "Результати D різні"
```

```
def compare_z(z_long, z_short):
    if z_long == z_short:
        return "Результати Z однакові"
    else:
        return "Результати Z різні"


# Основний файл програми

import random
import tkinter as tk
from tkinter import messagebox
import func_for_window2
import func_for_window3
import func_for_window4
import compare_rezult


class App:
    def __init__(self, app_root):
        self.root = app_root
        self.root.title("Вікно №1")
        self.root.geometry("600x600")

        self.group_number = 45
        self.number_list = 23
        self.window_2, self.window_3, self.window_4, self.window_5 = None, None,
None, None

        self.A = set()
        self.B = set()
        self.C = set()
        self.U = set()

        self.text = None

        # Головне меню
        menu_bar = tk.Menu(root)

        # Меню для відкриття інших вікон
        window_menu = tk.Menu(menu_bar, tearoff=0)
        menu_bar.add_cascade(label="Вікна", menu=window_menu, font=("Arial",
11))
        window_menu.add_command(label="Вікно № 2",
command=self.open_window2)
        window_menu.add_command(label="Вікно № 3",
```

```
command=self.open_window3)
        window_menu.add_command(label="Вікно № 4",
command=self.open_window4)
        window_menu.add_command(label="Вікно № 5",
command=self.open_window5)
        window_menu.configure(bg="#98ddff")

        root.config(menu=menu_bar)
        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)

        self.info_frame = tk.Frame(root, bg="white", bd=1, relief="solid")
        self.info_frame.grid(column=1, pady=20, padx=20)

        # Вивід П.І.Б, номер групи, номер у списку, результат
        self.name_label = tk.Label(self.info_frame, text="П.І.Б. студента: Унятицький
Антон Дмитрович", bg="white")
        self.name_label.grid(row=0, column=1, columnspan=1, sticky='e', padx=5,
pady=10)

        self.group_label = tk.Label(self.info_frame, text=f"Номер групи:
{self.group_number}", bg="white")
        self.group_label.grid(row=1, column=1, sticky='e', padx=5)

        self.list_number_label = tk.Label(self.info_frame, text=f"Номер у списку:
{self.number_list}", bg="white")
        self.list_number_label.grid(row=2, column=1, sticky='e', padx=5)

        self.variant_result_label = tk.Label(self.info_frame, text=f"Результат
обчислення варіанту: {self.generate_variant()}", bg="white")
        self.variant_result_label.grid(row=3, column=1, sticky='e', padx=5, pady=5)

        # Формування множин
        self.range_label = tk.Label(root, text="Діапазон цілих чисел (універсальна
множина):")
        self.range_label.configure(bg="#98ddff")
        self.range_label.grid(row=4, column=0, sticky='w', padx=10, pady=10)

        self.range_entry = tk.Entry(root)
        self.range_entry.grid(row=5, column=0, sticky='n', padx=10, pady=3)
        self.range_entry.configure(bg="#b7f0ff")
        self.range_entry.insert(0, "0, 20")

        self.univer_btn = tk.Button(root, text="Згенерувати",
command=self.generate_universal, bg="#bae5fb")
        self.univer_btn.grid(row=6, column=0, sticky='n', padx=10, pady=10)
```

```
        self.set_size_label = tk.Label(root, text="Потужність множин A, B, C:")
        self.set_size_label.configure(bg="#98ddff")
        self.set_size_label.grid(row=7, column=0, sticky='w', padx=10, pady=10)

        self.size_A_entry = tk.Entry(root)
        self.size_A_entry.configure(bg="#b7f0ff")
        self.size_A_entry.grid(row=8, column=0, sticky='n', padx=10, pady=3)
        self.size_A_entry.insert(0, "5")

        self.size_B_entry = tk.Entry(root)
        self.size_B_entry.grid(row=9, column=0, sticky='n', padx=10, pady=3)
        self.size_B_entry.configure(bg="#b7f0ff")
        self.size_B_entry.insert(0, "5")

        self.size_C_entry = tk.Entry(root)
        self.size_C_entry.grid(row=10, column=0, sticky='n', padx=10, pady=3)
        self.size_C_entry.configure(bg="#b7f0ff")
        self.size_C_entry.insert(0, "5")

        self.generate_button = tk.Button(root, text="Згенерувати множини A, B, C",
command=self.generate_sets, bg="#bae5fb")
        self.generate_button.grid(row=11, column=0, sticky='n', padx=10, pady=10)

        self.manual_input_label = tk.Label(root, text="Ввести множини вручну:")
        self.manual_input_label.configure(bg="#98ddff")
        self.manual_input_label.grid(row=12, column=0, sticky='w', padx=10, pady=10)

        self.manual_A_entry = tk.Entry(root)
        self.manual_A_entry.grid(row=13, column=0, sticky='n', padx=10, pady=3)
        self.manual_A_entry.configure(bg="#b7f0ff")

        self.manual_B_entry = tk.Entry(root)
        self.manual_B_entry.grid(row=14, column=0, sticky='n', padx=10, pady=3)
        self.manual_B_entry.configure(bg="#b7f0ff")

        self.manual_C_entry = tk.Entry(root)
        self.manual_C_entry.grid(row=15, column=0, sticky='n', padx=10, pady=3)
        self.manual_C_entry.configure(bg="#b7f0ff")

        self.save_btn = tk.Button(self.root, text="Зберегти", command=self.save_ch,
bg="#bae5fb")
        self.save_btn.grid(row=16, column=0, sticky='n', padx=10, pady=10)

    def generate_universal(self):
        try:
            min_ch, max_ch = map(int, self.range_entry.get().split(','))
            if min_ch > max_ch or min_ch < 0 or max_ch > 255:
```

```
            messagebox.showerror("Помилка", "Діапазон повинен бути між 0 і
255!")
            return
        self.U = set(range(min_ch, max_ch + 1))
        messagebox.showinfo("Універсальна множина", f"U: {self.U}")
    except ValueError:
        messagebox.showerror("Помилка", "Введіть два числа через кому!")

def open_window2(self):
    self.window_2 = tk.Toplevel(self.root)
    Window2(self.window_2, self.A, self.B, self.C, self.U)

def open_window3(self):
    self.window_3 = tk.Toplevel(self.root)
    Window3(self.window_3, self.A, self.B, self.C, self.U)

def open_window4(self):
    self.window_4 = tk.Toplevel(self.root)
    Window4(self.window_4, self.A, self.C, self.U)

def open_window5(self):
    self.window_5 = tk.Toplevel(self.root)
    Window5(self.window_5, self.A, self.C, self.U)

@staticmethod
def parse_set(text):
    try:
        return set(map(int, text.replace(",", " ").split()))
    except ValueError:
        messagebox.showerror("Помилка", "Введіть коректні числа через пробіл
або кому!")
        return None

def save_ch(self):
    try:
        size_a = int(self.size_A_entry.get())
        size_b = int(self.size_B_entry.get())
        size_c = int(self.size_C_entry.get())

        self.A = self.parse_set(self.manual_A_entry.get())
        self.B = self.parse_set(self.manual_B_entry.get())
        self.C = self.parse_set(self.manual_C_entry.get())

        if self.A is None or self.B is None or self.C is None:
            return
        if size_a == len(self.A) and size_b == len(self.B) and size_c == len(self.C):
            messagebox.showinfo("Збережені множини", f"A: {self.A}\nB:
```

```
{self.B}\nC: {self.C}")
        except ValueError:
            messagebox.showerror("Помилка", "Введіть коректні числові значення потужності множин!")

    def generate_sets(self):
        try:
            size_a = int(self.size_A_entry.get())
            size_b = int(self.size_B_entry.get())
            size_c = int(self.size_C_entry.get())
            min_u, max_u = map(int, self.range_entry.get().split(','))

            # Генерація множин
            self.A = set(random.sample(range(min_u, max_u), size_a))
            self.B = set(random.sample(range(min_u, max_u), size_b))
            self.C = set(random.sample(range(min_u, max_u), size_c))
            messagebox.showinfo("Згенеровані множини", f"A: {self.A}\nB: {self.B}\nC: {self.C}")
        except ValueError:
            messagebox.showerror("Помилка", "Введіть коректні числові значення потужності множин!")

    def generate_variant(self):
        rezult = (self.number_list + self.group_number % 60) % 30 + 1
        return rezult

class Window2:
    def __init__(self, window2_root, a, b, c, u):
        self.root = window2_root
        self.root.title("Вікно №2 - Виконання початкового виразу")
        self.root.geometry("600x500")
        self.root.configure(bg="#b7f0ff")
        self.A = a
        self.B = b
        self.C = c
        self.U = u
        self.result, self.result2, self.result3, self.result4, self.result5, self.result6, self.result7 = None, None, None, None, None, None, None

        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)

        self.mn = tk.Label(self.root, text="Усі множини", font="Arial 11")
        self.mn.configure(bg="#98ddff")
        self.mn.grid(row=0, column=0, sticky='s', padx=5, pady=10)
        self.n_c = func_for_window2.not_c_f(self.U, self.C)
```

```
        self.initial_u = tk.Label(self.root, text=f"U: {self.U}", font="Arial 11")
        self.initial_u.configure(bg="#98ddff")
        self.initial_u.grid(row=1, column=0, sticky='w', padx=5, pady=2)
        self.initial_a = tk.Label(self.root, text=f"A: {self.A}", font="Arial 11")
        self.initial_a.configure(bg="#98ddff")
        self.initial_a.grid(row=2, column=0, sticky='w', padx=5, pady=2)
        self.initial_b = tk.Label(self.root, text=f"B: {self.B}", font="Arial 11")
        self.initial_b.configure(bg="#98ddff")
        self.initial_b.grid(row=3, column=0, sticky='w', padx=5, pady=2)
        self.initial_c = tk.Label(self.root, text=f"C: {self.C}", font="Arial 11")
        self.initial_c.configure(bg="#98ddff")
        self.initial_c.grid(row=4, column=0, sticky='w', padx=5, pady=2)
        self.initial_not_c = tk.Label(self.root, text=f"c̄ : {self.n_c}", font="Arial 11")
        self.initial_not_c.configure(bg="#98ddff")
        self.initial_not_c.grid(row=5, column=0, sticky='w', padx=5, pady=2)
        self.initial_d = tk.Label(self.root, text=f"D: (((A ∪ Ā)∩ A ) \\ (B ∪ B)) ∩
(C̄∪(C̄∩B̄))", font="Arial 11")
        self.initial_d.configure(bg="#98b6ff")
        self.initial_d.grid(row=6, column=0, sticky='w', padx=5, pady=2)

        self.step_button = tk.Button(self.root, text="Крок 1", command=self.step_1_op,
bg="#bae5fb")
        self.step_button.grid(row=7, column=0, sticky='w', padx=10, pady=2)

        self.step2_button = tk.Button(self.root, text="Крок 2",
command=self.step_2_op, bg="#bae5fb")
        self.step2_button.grid(row=8, column=0, sticky='w', padx=10, pady=2)

        self.step3_button = tk.Button(self.root, text="Крок 3",
command=self.step_3_op, bg="#bae5fb")
        self.step3_button.grid(row=9, column=0, sticky='w', padx=10, pady=2)

        self.step4_button = tk.Button(self.root, text="Крок 4",
command=self.step_4_op, bg="#bae5fb")
        self.step4_button.grid(row=10, column=0, sticky='w', padx=10, pady=2)

        self.step5_button = tk.Button(self.root, text="Крок 5",
command=self.step_5_op, bg="#bae5fb")
        self.step5_button.grid(row=11, column=0, sticky='w', padx=10, pady=2)

        self.step6_button = tk.Button(self.root, text="Крок 6",
command=self.step_6_op, bg="#bae5fb")
        self.step6_button.grid(row=12, column=0, sticky='w', padx=10, pady=2)

        self.step7_button = tk.Button(self.root, text="Крок 7",
command=self.step_7_op, bg="#bae5fb")
```

```
        self.step7_button.grid(row=13, column=0, sticky='w', padx=10, pady=2)

        self.save_button = tk.Button(self.root, text="Зберегти D у файл",
command=self.save_result, bg="#bae5fb")
        self.save_button.grid(row=14, column=0, sticky='w', padx=10, pady=10)

    def step_1_op(self):
        self.result = func_for_window2.step_1_f(self.A,
func_for_window2.not_a_f(self.U, self.A))
        tk.Label(self.root, text=f"A ∪ Ā: {self.result}",
                 font="Arial 11", background="#98ddff").grid(row=7, column=0,
sticky='w', padx=60, pady=2)
        return self.result

    def step_2_op(self):
        self.result2 = func_for_window2.step_2_f(self.step_1_op(), self.A)
        tk.Label(self.root, text=f"(A ∪ Ā)∩A: {self.result2}",
                 font="Arial 11", background="#98ddff").grid(row=8, column=0,
sticky='w', padx=60, pady=2)
        return self.result2

    def step_3_op(self):
        self.result3 = func_for_window2.step_3_f(self.B)
        tk.Label(self.root, text=f"B ∪ B: {self.result3}",
                 font="Arial 11", background="#98ddff").grid(row=9, column=0,
sticky='w', padx=60, pady=2)
        return self.result3

    def step_4_op(self):
        self.result4 = func_for_window2.step_4_f(self.step_2_op(),
func_for_window2.not_b_f(self.U, self.B))
        tk.Label(self.root, text=f"((A ∪ Ā)∩A)\\(B ∪ B): {self.result4}",
                 font="Arial 11", background="#98ddff").grid(row=10, column=0,
sticky='w', padx=60, pady=2)
        return self.result4

    def step_5_op(self):
        self.result5 = func_for_window2.step_5_f(self.C, self.step_3_op())
        tk.Label(self.root, text=f"C ∩ B: {self.result5}",
                 font="Arial 11", background="#98ddff").grid(row=11, column=0,
sticky='w', padx=60, pady=2)
        return self.result5

    def step_6_op(self):
        self.result6 = func_for_window2.step_6_f(self.C, self.step_5_op())
        tk.Label(self.root, text=f"C ∪ (C ∩ B): {self.result6}",
```

```
            font="Arial 11", background="#98ddff").grid(row=12, column=0,
sticky='w', padx=60, pady=2)
        return self.result6

    def step_7_op(self):
        self.result7 = func_for_window2.step_7_f(self.step_4_op(),
func_for_window2.not_c_f(self.U, self.C))
        tk.Label(self.root, text=f"((A ∪ Ā)∩A)\\(B ∪ B) ∩ (C ∪ (C ∩ B)):
{self.result7}",
            font="Arial 11", background="#98ddff").grid(row=13, column=0,
sticky='w', padx=60, pady=2)
        return self.result7

    def save_result(self):
        with open("result.txt", "w") as f:
            d = self.step_7_op()
            f.write(str(d))
        tk.Label(self.root, text="Збережено!", font="Arial 14",
background="#98ddff").grid(row=14, column=0, sticky='w', padx=150, pady=2)
        return

class Window3:
    def __init__(self, window3_root, a, b, c, u):
        self.root = window3_root
        self.root.title("Вікно №3 - Виконання спрощеного виразу")
        self.root.geometry("600x350")
        self.root.configure(bg="#b7f0ff")
        self.a, self.b, self.c, self.u = a, b, c, u
        self.step_1s, self.step_2s = None, None

        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)

        tk.Label(self.root, text="Усі множини", font="Arial 11",
            background="#98b6ff").grid(row=0, column=0, sticky='s', padx=5,
pady=10)
        self.n_c = func_for_window2.not_c_f(self.u, self.c)

        self.initial_u = tk.Label(self.root, text=f"U- {self.u}", font="Arial 11",
background="#98ddff")
        self.initial_u.grid(row=1, column=0, sticky='w', padx=5, pady=2)
        self.initial_a = tk.Label(self.root, text=f"A- {self.a}", font="Arial 11",
background="#98ddff")
        self.initial_a.grid(row=2, column=0, sticky='w', padx=5, pady=2)
        self.initial_b = tk.Label(self.root, text=f"B- {self.b}", font="Arial 11",
background="#98ddff")
```

```
        self.initial_b.grid(row=3, column=0, sticky='w', padx=5, pady=2)
        self.initial_c = tk.Label(self.root, text=f"C- {self.c}", font="Arial 11",
background="#98ddff")
        self.initial_c.grid(row=4, column=0, sticky='w', padx=5, pady=2)
        self.initial_not_c = tk.Label(self.root, text=f"c̄ - {self.n_c}", font="Arial 11",
background="#98ddff")
        self.initial_not_c.grid(row=5, column=0, sticky='w', padx=5, pady=2)
        self.initial_d = tk.Label(self.root, text=f"D: A ∪ B̄ ∩ C̄", font="Arial 11",
background="#98ddff")
        self.initial_d.grid(row=6, column=0, sticky='w', padx=5, pady=2)

        self.step1_sh_btn = tk.Button(self.root, text="Крок 1", command=self.step_1sh,
bg="#bae5fb")
        self.step1_sh_btn.grid(row=8, column=0, sticky='w', padx=5, pady=2)

        self.step2_sh_btn = tk.Button(self.root, text="Крок 2", command=self.step_2sh,
bg="#bae5fb")
        self.step2_sh_btn.grid(row=9, column=0, sticky='w', padx=5, pady=2)

        self.save_button = tk.Button(self.root, text="Зберегти D у файл",
command=self.save_result, bg="#bae5fb")
        self.save_button.grid(row=10, column=0, sticky='w', padx=10, pady=10)

    def step_1sh(self):
        self.step_1s = func_for_window3.step1_f_sh(self.a,
func_for_window3.not_b_f(self.u, self.b))
        tk.Label(self.root, text=f"A ∪ B̄ -> {self.step_1s}",
                 background="#98ddff").grid(row=8, column=0, sticky='w', padx=65,
pady=2)
        return self.step_1s

    def step_2sh(self):
        self.step_2s = func_for_window3.step2_f_sh(self.step_1sh(), self.n_c)
        tk.Label(self.root, text=f" A ∪ B̄ ∩ C̄ -> {self.step_2s}",
                 background="#98ddff").grid(row=9, column=0, sticky='w', padx=65,
pady=2)
        return self.step_2s

    def save_result(self):
        with open("result_s.txt", "w") as f:
            d = self.step_2sh()
            f.write(str(d))
        tk.Label(self.root, text="Збережено!", font="Arial 14",
                 background="#98ddff").grid(row=10, column=0, sticky='w', padx=150,
pady=2)
        return
```

```
class Window4:
    def __init__(self, window4_root, a, c, u):
        self.root = window4_root
        self.root.title("Вікно №4")
        self.root.geometry("600x300")
        self.root.configure(bg="#b7f0ff")
        self.x, self.y, self.u = a, c, u
        self.step_v = None

        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)

        tk.Label(self.root, text="Множини для виразу", font="Arial 11",
                 background="#98b6ff").grid(row=0, column=0, sticky='s', padx=5,
pady=10)
        self.initial_x = tk.Label(self.root, text=f"X- {self.x}", font="Arial 11",
background="#98ddff")
        self.initial_x.grid(row=1, column=0, sticky='w', padx=5, pady=2)
        self.initial_y = tk.Label(self.root, text=f"Y- {self.y}", font="Arial 11",
background="#98ddff")
        self.initial_y.grid(row=2, column=0, sticky='w', padx=5, pady=2)
        self.initial_z = tk.Label(self.root, text=f"Z = X \\ Y ", font="Arial 11",
background="#98ddff")
        self.initial_z.grid(row=3, column=0, sticky='w', padx=5, pady=2)

        self.step_btn = tk.Button(self.root, text="Крок", font="Arial 11",
command=self.step_s, bg="#bae5fb")
        self.step_btn.grid(row=4, column=0, sticky='w', padx=5, pady=2)

        self.save_button = tk.Button(self.root, text="Зберегти Z у файл",
command=self.save_result, bg="#bae5fb")
        self.save_button.grid(row=6, column=0, sticky='w', padx=10, pady=10)

    def step_s(self):
        self.step_v = func_for_window4.my_func(self.x, self.y, self.u)
        tk.Label(self.root, text=f"Z = X \\ Y => {self.step_v}",
                 background="#98ddff").grid(row=4, column=0, sticky='w', padx=65,
pady=2)
        return self.step_v

    def save_result(self):
        with open("result_z1.txt", "w") as f:
            z = self.step_s()
            f.write(str(z))
        tk.Label(self.root, text="Збережено!", font="Arial 14",
                 background="#98ddff").grid(row=6, column=0, sticky='w', padx=150,
```

```
pady=2)
        return

class Window5:
    def __init__(self, window5_root, a, c, u):
        self.root = window5_root
        self.root.title("Вікно №5")
        self.root.geometry("600x400")
        self.root.configure(bg="#b7f0ff")
        self.x, self.y, self.u = a, c, u

        self.file1_text = None
        self.file2_text = None
        self.file3_text = None
        self.file4_text = None
        self.res, self.res_z = None, None

        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)

        self.file1_btn = tk.Button(self.root, text="Зчитати D початковий",
command=self.file1_func,
                                   font="Arial 11", bg="#bae5fb")
        self.file1_btn.grid(row=1, column=0, sticky='w', padx=5, pady=5)

        self.file2_btn = tk.Button(self.root, text="Зчитати D скорочений",
command=self.file2_func,
                                   font="Arial 11", bg="#bae5fb")
        self.file2_btn.grid(row=2, column=0, sticky='w', padx=5, pady=5)

        self.btn_com = tk.Button(self.root, text="Порівняти",
                                 font="Arial 11", command=self.compare_text, bg="#bae5fb")
        self.btn_com.grid(row=3, column=0, sticky='w', padx=5, pady=5)

        self.file3_btn = tk.Button(self.root, text="Зчитати Z (моя функція)",
                                   font="Arial 11", command=self.file3_func, bg="#bae5fb")
        self.file3_btn.grid(row=5, column=0, sticky='w', padx=5, pady=5)

        self.file4_btn = tk.Button(self.root, text="Зчитати Z (функція python)",
                                   font="Arial 11", command=self.file4_func, bg="#bae5fb")
        self.file4_btn.grid(row=6, column=0, sticky='w', padx=5, pady=5)

        self.comp_z = tk.Button(self.root, text="Порівняти",
                                font="Arial 11", command=self.z_comp, bg="#bae5fb")
        self.comp_z.grid(row=7, column=0, sticky='w', padx=5, pady=5)

    def compare_text(self):
```

```
        self.res = compare_rezult.compare(self.file1_text, self.file2_text)
        tk.Label(self.root, text=f"{self.res}", font="Arial 11",
                 background="#98ddff"). grid(row=3, column=0, sticky='w', padx=100,
pady=5)
        return self.res

    def file1_func(self):
        with open("result.txt", "r") as f:
            self.file1_text = f.read()
        tk.Label(self.root, text=f"{self.file1_text}", font="Arial 11",
                 background="#98ddff").grid(row=1, column=0, sticky='w', padx=200,
pady=5)
        return

    def file2_func(self):
        with open("result_s.txt", "r") as f:
            self.file2_text = f.read()
        tk.Label(self.root, text=f"{self.file2_text}", font="Arial 11",
                 background="#98ddff").grid(row=2, column=0, sticky='w', padx=200,
pady=5)
        return

    def file3_func(self):
        with open("result_z1.txt", "r") as f:
            self.file3_text = f.read()
        tk.Label(self.root, text=f"{self.file3_text}", font="Arial 11",
                 background="#98ddff").grid(row=5, column=0, sticky='w', padx=200,
pady=5)
        return

    def file4_func(self):
        self.file4_text = func_for_window4.func_python(self.x, self.y, self.u)
        with open("result_z2.txt", "w") as f:
            z = self.file4_text
            f.write(str(z))

        with open("result_z2.txt", "r") as f:
            self.file4_text = f.read()
        tk.Label(self.root, text=f"{self.file4_text}", font="Arial 11",
background="#98ddff").grid(
                row=6, column=0, sticky='w', padx=200,pady=5)
        return self.file4_text

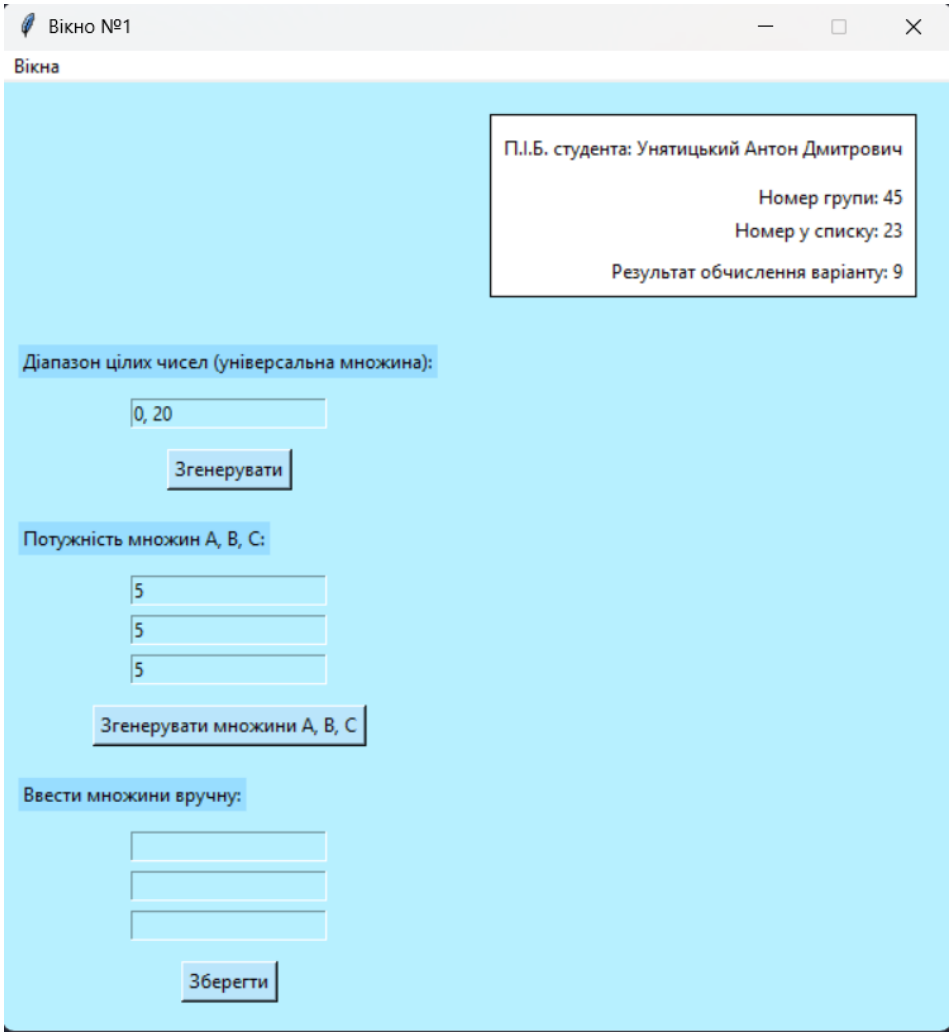
    def z_comp(self):
        self.res_z = compare_rezult.compare_z(self.file3_text, self.file4_text)
        tk.Label(self.root, text=f"{self.res_z}", font="Arial 11",
                 background="#98ddff").grid(row=7, column=0, sticky='w', padx=100,
```

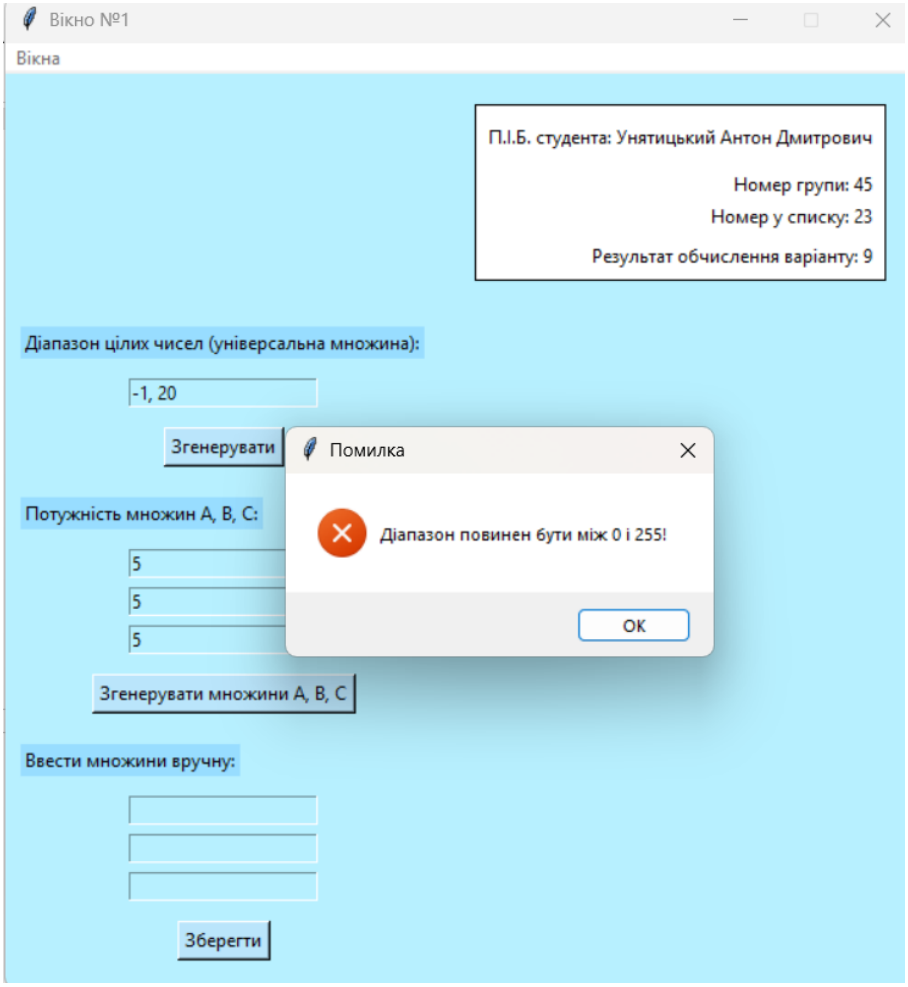
```
pady=5)
        return self.res_z

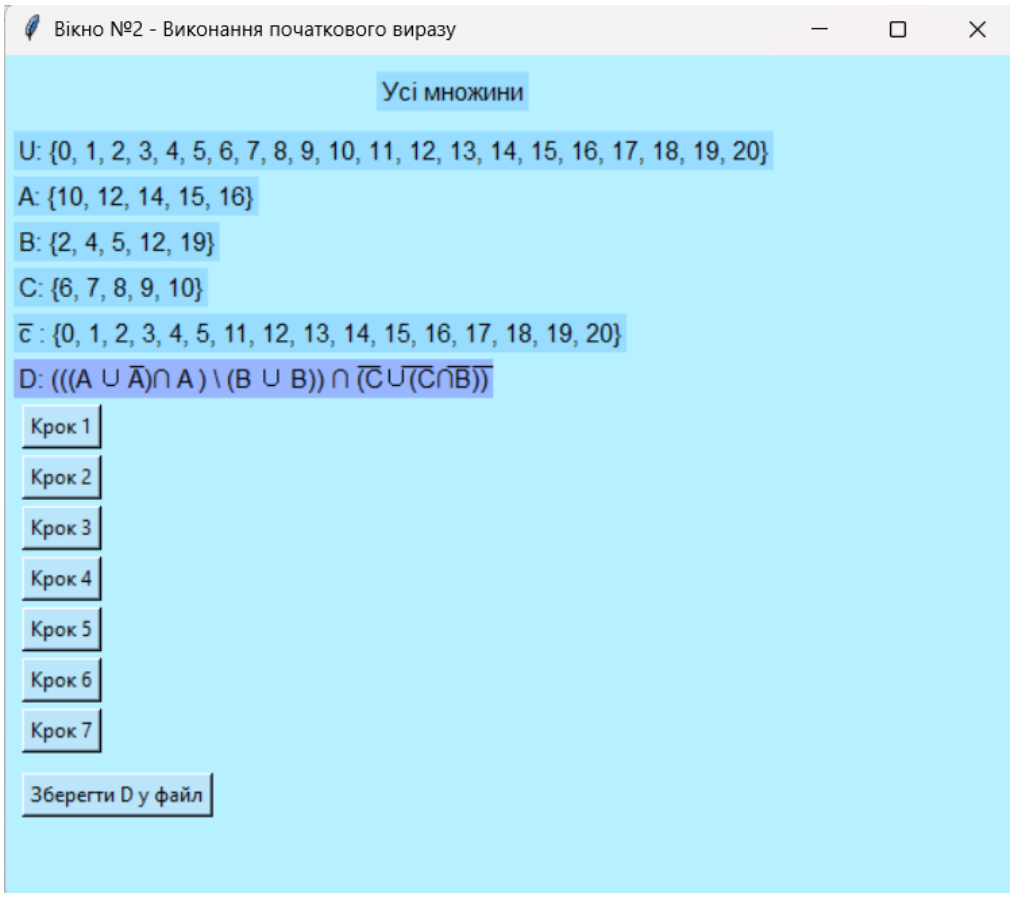
if __name__ == "__main__":
    root = tk.Tk()
    root.resizable(False, False)
    root.protocol("WM_DELETE_WINDOW", root.quit)
    root.configure(bg="#b7f0ff")
    app = App(root)
    root.mainloop()
```

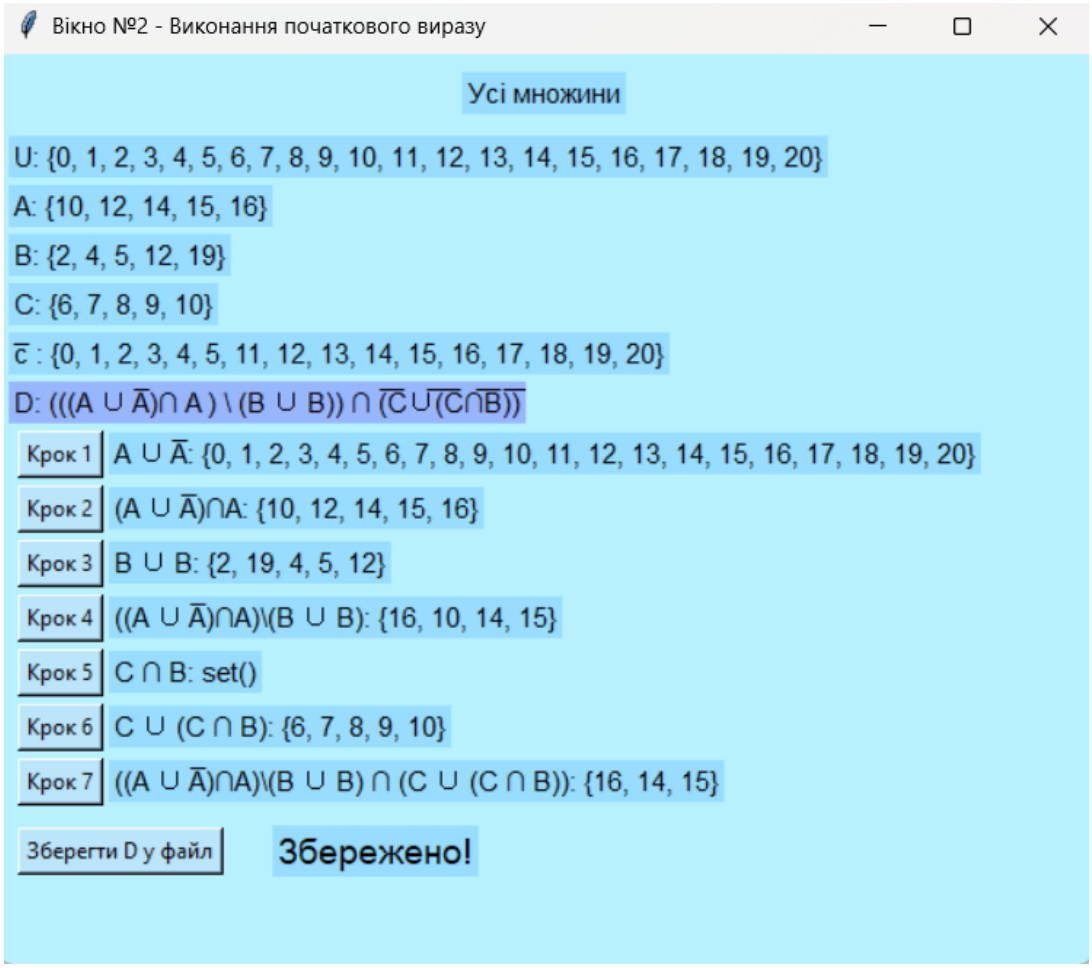
Скріншоти виконання програми:

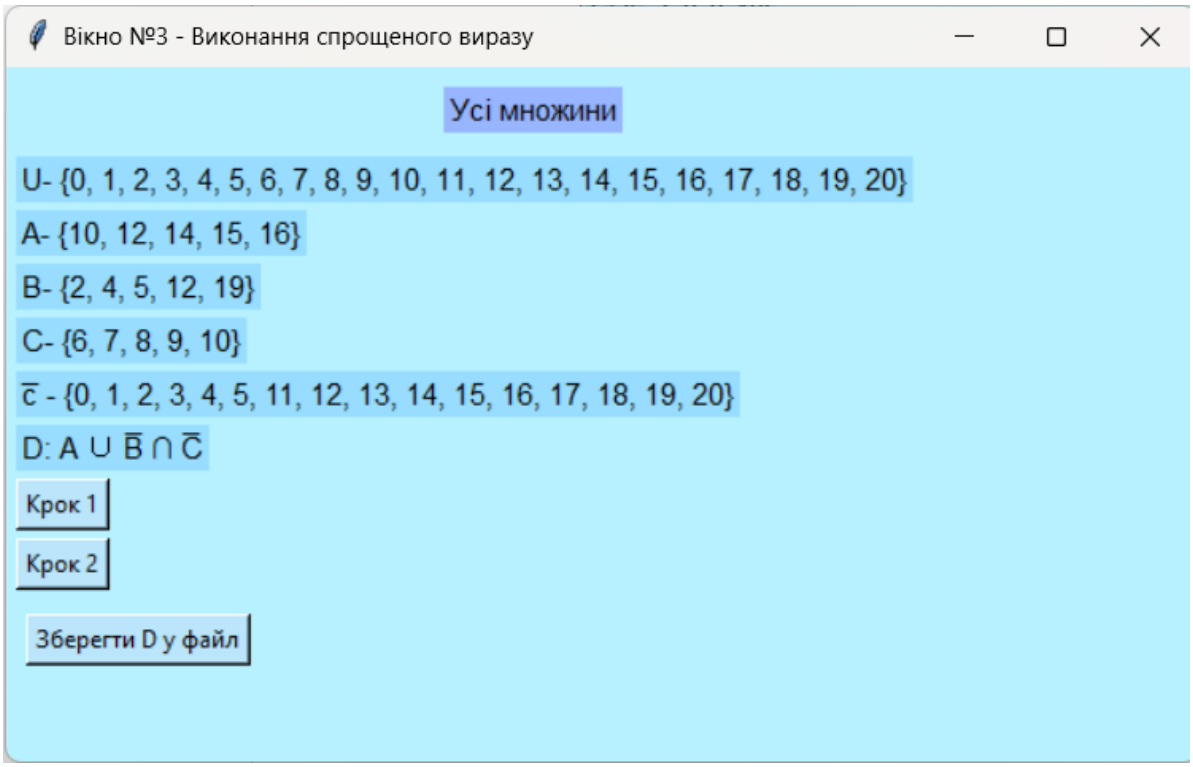
Приклад ручного введення

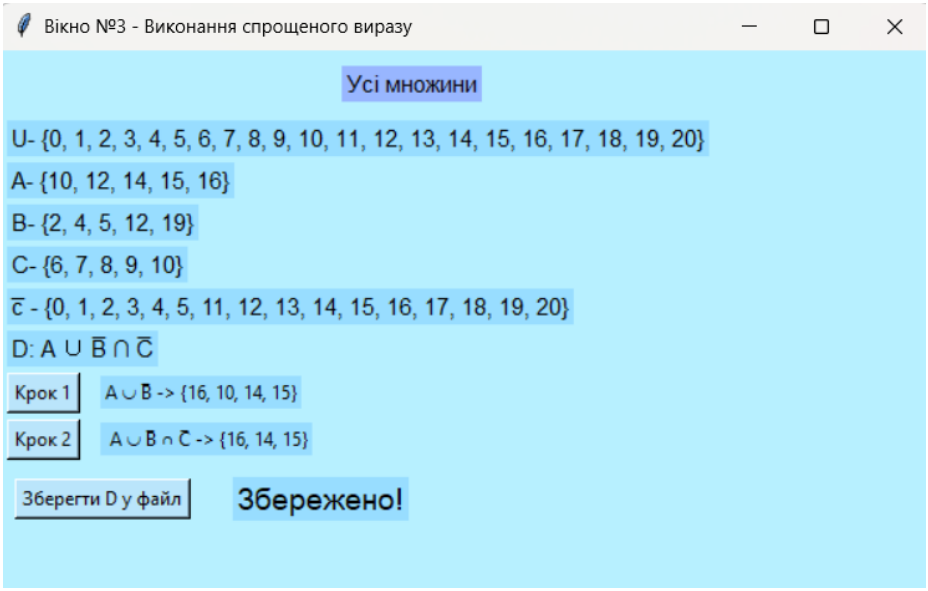


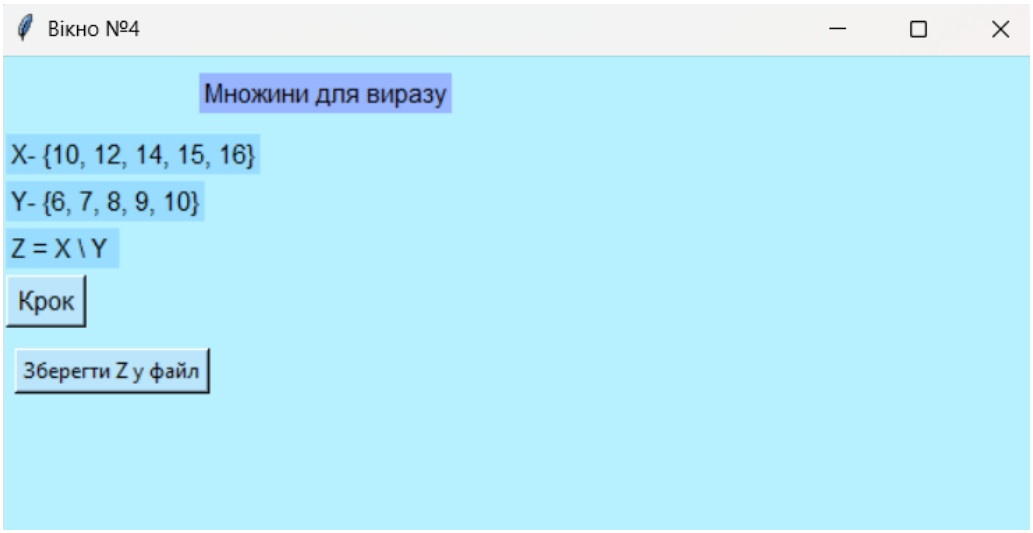

Вікно №1
Вікна
П.І.Б. студента: Унятицький Антон Дмитрович
Номер групи: 45
Номер у списку: 23
Результат обчислення варіанту: 9
Діапазон цілих чисел (універсальна множина):
-1, 20
Згенерувати
Помилка
Діапазон повинен бути між 0 і 255!
OK
Потужність множин A, B, C:
5
5
5
Згенерувати множини A, B, C
Ввести множини вручну:
Зберегти

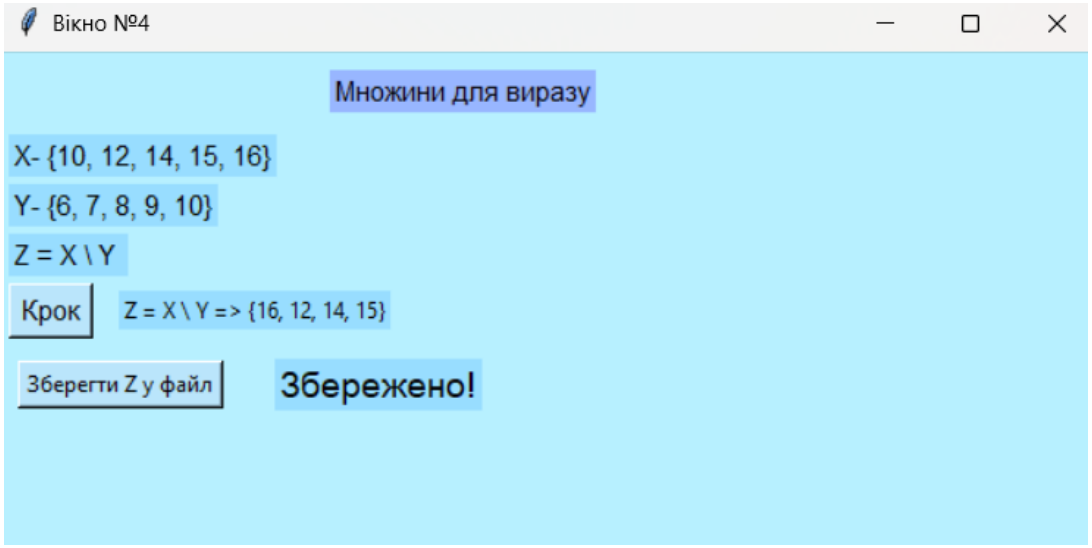

Вікно №2 - Виконання початкового виразу
Усі множини
U: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
A: {10, 12, 14, 15, 16}
B: {2, 4, 5, 12, 19}
C: {6, 7, 8, 9, 10}
c̄ : {0, 1, 2, 3, 4, 5, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
D: (((A ∪ Ā)∩ A) \ (B ∪ B)) ∩ (C̄ ∪ (C∩B̄))
Крок 1
Крок 2
Крок 3
Крок 4
Крок 5
Крок 6
Крок 7
Зберегти D у файл

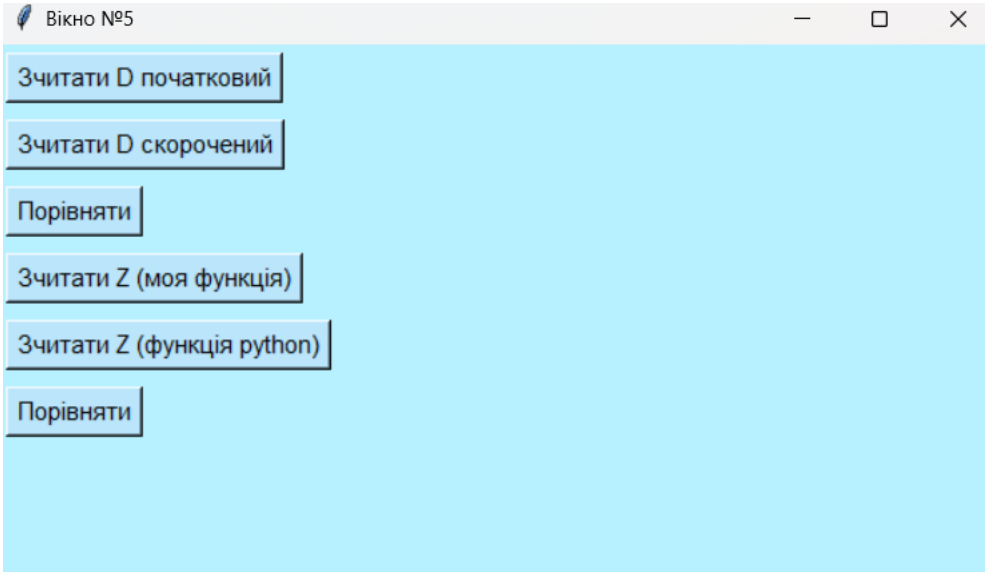

Вікно №2 - Виконання початкового виразу
Усі множини
U: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
A: {10, 12, 14, 15, 16}
B: {2, 4, 5, 12, 19}
C: {6, 7, 8, 9, 10}
$\overline{c}$: {0, 1, 2, 3, 4, 5, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
D: (((A ∪ $\overline{A}$)∩ A) \ (B ∪ B)) ∩ ($\overline{C}$ ∪ $\overline{(C \cap B)}$)
Крок 1
A ∪ $\overline{A}$: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
Крок 2
(A ∪ $\overline{A}$)∩A: {10, 12, 14, 15, 16}
Крок 3
B ∪ B: {2, 19, 4, 5, 12}
Крок 4
((A ∪ $\overline{A}$)∩A)\(B ∪ B): {16, 10, 14, 15}
Крок 5
C ∩ B: set()
Крок 6
C ∪ (C ∩ B): {6, 7, 8, 9, 10}
Крок 7
((A ∪ $\overline{A}$)∩A)\(B ∪ B) ∩ (C ∪ (C ∩ B)): {16, 14, 15}
Зберегти D у файл
Збережено!

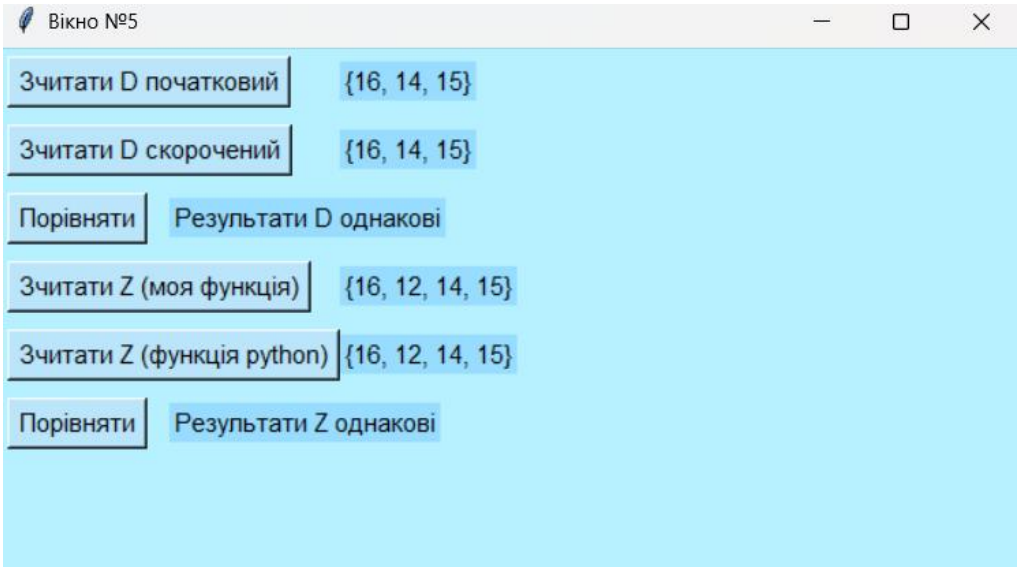

Вікно №3 - Виконання спрощеного виразу
Усі множини
U- {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
A- {10, 12, 14, 15, 16}
B- {2, 4, 5, 12, 19}
C- {6, 7, 8, 9, 10}
$\overline{c}$ - {0, 1, 2, 3, 4, 5, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
D: A ∪ $\overline{B}$ ∩ $\overline{C}$
Крок 1
Крок 2
Зберегти D у файл

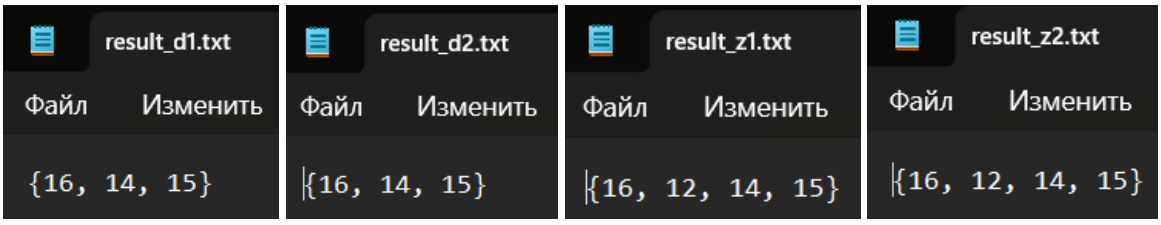

Вікно №3 - Виконання спрощеного виразу
Усі множини
U- {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
A- {10, 12, 14, 15, 16}
B- {2, 4, 5, 12, 19}
C- {6, 7, 8, 9, 10}
$\overline{C}$ - {0, 1, 2, 3, 4, 5, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
D: A ∪ $\overline{B}$ ∩ $\overline{C}$
Крок 1
A ∪ $\overline{B}$ -> {16, 10, 14, 15}
Крок 2
A ∪ $\overline{B}$ ∩ $\overline{C}$ -> {16, 14, 15}
Зберегти D у файл
Збережено!


Вікно №4
Множини для виразу
X- {10, 12, 14, 15, 16}
Y- {6, 7, 8, 9, 10}
Z = X \ Y
Крок
Зберегти Z у файл


Вікно №4
Множини для виразу
X- {10, 12, 14, 15, 16}
Y- {6, 7, 8, 9, 10}
Z = X \ Y
Крок
Z = X \ Y => {16, 12, 14, 15}
Зберегти Z у файл
Збережено!







Висновки: На цій лабораторній роботі я повторив бібліотеку tkinter, вивчив основні операції над множинами, застосував їх на практиці при обрахунку свого логічного виразу в індивідуальному завданні.

Я написав програму, яка складається з 5 вікон , де 1 вікно – головне й містить виклики до інших вікон. Програма за допомогою введеня даних в головному вікні проводить покрокові обчислення виразів D і Z і в кінці порівнює вирази результатів D-початкового й D-спрощеного, а також Z написаного моєю функцією й Z – функцією мови python. По результатах тестування програми можна побачити , що результати співпадають , тобто обчислення над множинами виконано вірно.